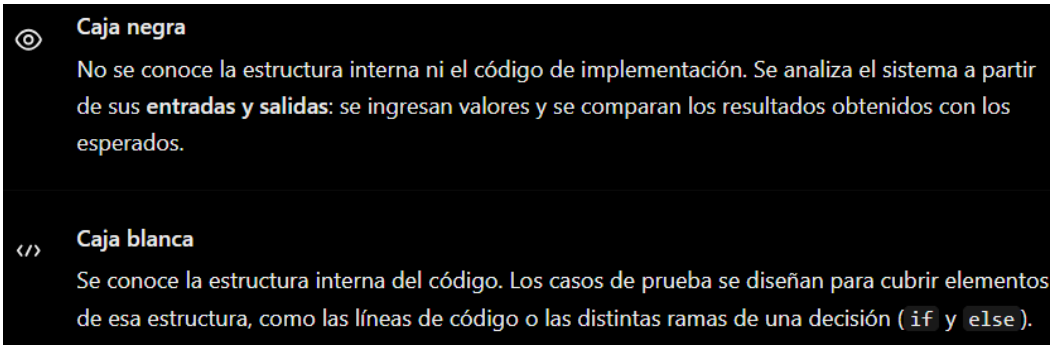
TESTING CAJA NEGRA

¿Para qué sirven las estrategias de testing?

El tiempo y el presupuesto son limitados, por eso se utilizan estrategias que permiten aprovechar mejor el tiempo y el presupuesto disponibles. El objetivo es **probar la mayor cantidad de funcionalidades posible con la menor cantidad de casos de prueba.**

Es decir, las estrategias buscan la mayor cobertura al menor costo, donde "costo" es la cantidad de casos de prueba. Los casos se identifican según los **criterios de aceptación**.

Dentro de las estrategias de testing se distinguen dos enfoques principales:



Lo ideal es **combinar ambas**: caja blanca para algunas cosas y caja negra para otras. Cada uno permite detectar tipos de defectos diferentes.

En el práctico, veremos Caja Negra.

MÉTODOS DE CAJA NEGRA

Existen 2 métodos:

- **Basados en especificaciones:** partición de equivalencias y análisis de valores límites, entre otros.
- **Basados en la experiencia:** adivinanza de defectos y testing exploratorio.

Ningún método es el mejor por sí solo. Cada uno tiene fortalezas y debilidades, y **usar una variedad de técnicas** da un testing más efectivo.

En el práctico, vemos las dos técnicas basadas en especificaciones.

Partición de equivalencias

Idea: analizar las condiciones externas de una funcionalidad y agrupar sus valores posibles en subconjuntos que producen un resultado **equivalente (no igual).**

Clase de equivalencia: un subconjunto de valores de una condición externa tal que cualquier miembro produce un resultado equivalente. Por eso alcanza con probar **un solo representante**.

La idea es que no hace falta probar todos los valores posibles: si varios valores se comportan de manera equivalente, se selecciona un representante de esa clase.
Por eso, la técnica permite reducir la cantidad de casos de prueba sin perder de vista los comportamientos que deben verificarse.

Tipos de clases:

- Rango de valores continuos
- Valores discretos
- Selección simple (elegir una ciudad de una lista)
- Selección múltiple (elegir más de una)

Pasos:

1. Definir las **condiciones externas de entrada** (un dato ingresado por el usuario, una opción seleccionada, la edad, el saldo de una tarjeta, etc.).

2. Definir las **clases de equivalencia de cada entrada**, válidas e inválidas.

   Clase válida: contiene valores que cumplen las condiciones establecidas y permiten el comportamiento esperado.

   Clase inválida: contiene valores que no cumplen las condiciones establecidas y deberían producir un rechazo, una validación o un error.

   Por ejemplo, para una funcionalidad que permite ingresar a un sitio de bebidas alcohólicas según la edad, las clases podrían ser:

Clase	Valores posibles	Resultado esperado
Válida	Enteros desde 18 hasta 100, según la regla del ejemplo	Permite ingresar
Inválida	Enteros mayores que 0 y menores que 18	No permite ingresar
Inválida	Enteros menores que 0 o mayores que 100	Valor inválido
Inválida	Caracteres no numéricos	Valor inválido

   Una misma condición externa puede tener varias clases de equivalencia inválidas. Por ejemplo, ingresar letras, dejar el campo vacío o ingresar un número decimal pueden requerir validaciones y mensajes de error diferentes. Si el comportamiento esperado cambia, corresponde considerar clases diferentes.

3. Definir las **condiciones externas de salida** y sus clases. Todo lo que devuelve el software es candidato: mensajes de éxito, mensajes de error, mails, etc. Los mensajes de error son siempre clases inválidas.

4. **Armar los casos de prueba** cubriendo todas las clases.

Plantilla:

Campo	Qué es
Id	Número que lo identifica
Prioridad	Alta, media o baja. Se prioriza lo que permite encontrar más defectos y garantiza la "salud" de la funcionalidad
Nombre	Describe claramente el escenario probado
Clases (CE)	Números de las clases de equivalencia que incluye
Precondiciones	Estado del contexto necesario, con valores **específicos** (ej.: "El usuario 'juan' está logueado con permisos de administrador", "La fecha actual es 15/5/2020")
Pasos	Operaciones ordenadas y numeradas, redactadas como "El ROL hace tal cosa"
Resultado esperado	Lo que muestra o produce el sistema. Puede haber más de una salida

Criterio de prioridad:

- **Alta:** caminos felices, que sí o sí deben funcionar.
- **Media:** combinaciones de valores que pueden producir una falla o un camino no feliz.
- **Baja:** validaciones, valores no ingresados o con formato incorrecto, porque los defectos ahí son menos frecuentes.

Análisis de valores límites

Es una **variante de la partición de equivalencias**. Se basa en que la mayor cantidad de defectos aparece en los **extremos de los intervalos**.

Su fundamento es que muchos defectos aparecen en los límites de los rangos.
En lugar de elegir cualquier valor representativo de una clase, se eligen valores próximos a sus límites para comprobar el comportamiento del sistema.

¿Qué diferencia hay con la partición de equivalencias?

- Las **clases se definen igual**; lo que cambia son los **casos de prueba**.
- En vez de un valor cualquiera de la clase, se prueba **un valor justo en el límite (dentro) y otro justo fuera**, en cada límite. Ejemplo (edad ≥ 18 y < 100): se prueban 17, 18, 99 y 100.
- Genera **más casos** que la partición simple, que prueba un solo valor por clase.
- Los límites numéricos son claros, pero también existen en otros casos no tan sencillos.

El análisis de valores límites no se restringe a valores numéricos continuos: lo importante es identificar dónde cambia el comportamiento esperado. Por ejemplo para un DNI, se puede considerar la cantidad de caracteres permitida.